

Software Stack & Implementation Guide

What was built for the smart assistive walker, what the AI models can and cannot do, and exactly how the team turns this into a working prototype.

Repository	github.com/sayansaunth-lang/sensewalk
Hardware in hand	ESP32-S3, Raspberry Pi 3B (1 GB), Pi Camera Module 3
Software status	Complete and tested on a laptop (163 automated tests pass)
Hardware status	Not built yet. Firmware compiles (checked on GitHub) but has never run on a real board. Thresholds are placeholders
Own AI model	YOLOv8n pothole detector: AP50 0.51, precision 0.64, recall 0.49 (92-image held-out test)

How to read this document. Every number here was measured on a real run, and every claim that was *not* tested is labelled as such. Section 7 lists exactly what has and has not been verified, so nobody on the team assumes more than is true.

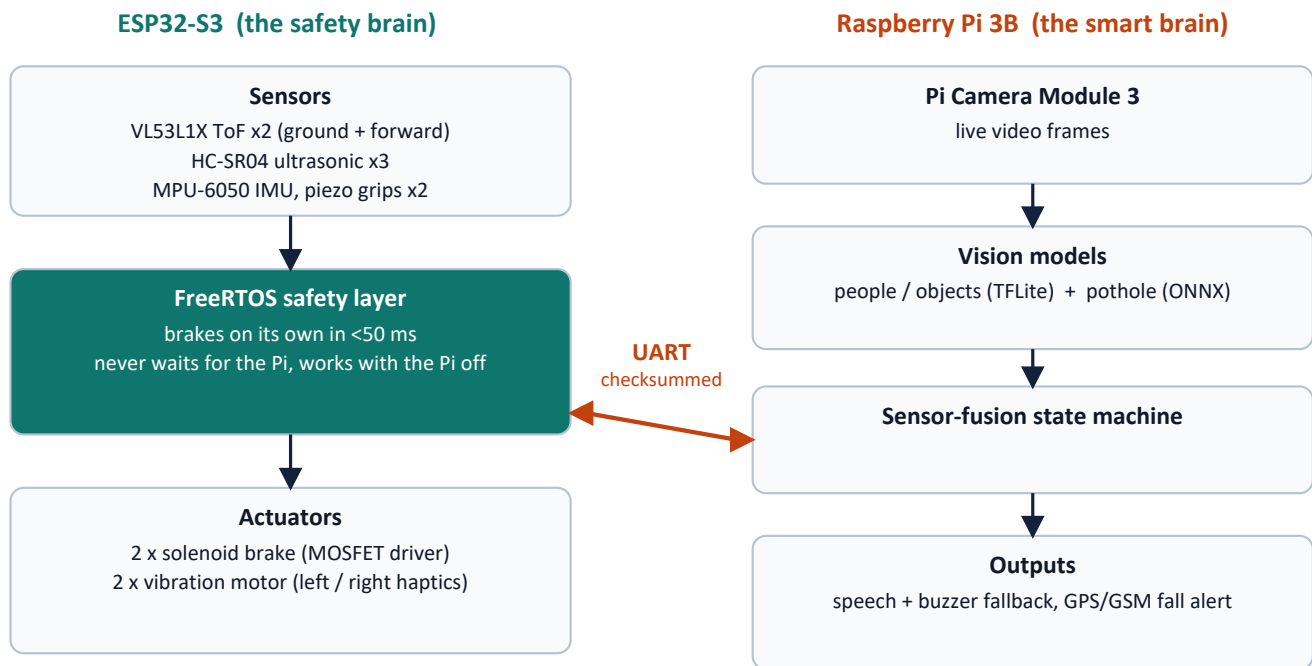
1. Where the project stands

SENSEWALK is a low-cost walker for visually impaired users. Its design splits the work between two processors: an **ESP32-S3** that senses hazards and brakes by itself in under 50 ms, and a **Raspberry Pi** that adds vision, speech and emergency messaging. The software for both is written, structured per team member, and pushed to GitHub. What does not exist yet is the physical walker.

Area	Status	Evidence / caveat
Pi-side software (vision, fusion, speech, UART)	Done, tested	163 automated tests pass on a laptop
People / object detector	Verified with real weights	Run on real photos; finds people, also makes some false detections (section 4.1)
Fall -> SMS with GPS location (emergency/)	Built, tested on a fake modem	Never run on a real SIM800L / NEO-6M; needs a 2G network. Use the bench script first (section 6)
Pothole detector (our own model)	Trained, modest accuracy	Finds about half of potholes; about 1 in 3 detections is wrong (section 4.2)
ESP32 firmware (Arduino IDE + PlatformIO)	Compiles, never flashed	Both versions build on GitHub with the latest ESP32 core and its 19 C++ tests pass, but it has never run on a real board
Sensor thresholds (drop-off, obstacle, fall, grip)	Placeholders	Marked TODO(calibrate); must be replaced with bench measurements
Pi 3B real-world speed	Not measured	Only laptop timings exist; the Pi estimate is an extrapolation
Chassis, wiring, brakes, power	Not started	Mechanical track (M1, M2)
Automatic checks (CI) on GitHub	Live, all green	4 jobs on every push: Python tests, PlatformIO firmware build, Arduino IDE sketch build, firmware C++ tests

The most important design rule. The camera cannot measure distance, so vision is only ever allowed to **warn** (speech and vibration). Only the ESP32's time-of-flight and ultrasonic sensors, which measure real distance, can fire the brake. A slow, crashed or wrong Pi therefore cannot cause a wrong brake decision, and the walker stays safe even if every AI model is switched off.

2. How the system fits together



Why two processors? Linux on the Pi can pause for tens of milliseconds at random, which is unacceptable for a brake. The ESP32 runs a real-time operating system (FreeRTOS) with the safety loop at the highest priority, so its reaction time is predictable. The Pi has the memory and speed for cameras and neural networks, but is never trusted with the brake.

The two links between them

- **UART cable (115200 baud).** The ESP32 streams distances, grip state and a heartbeat to the Pi; the Pi sends back haptic commands. Every line carries a sequence number and an XOR checksum, so corrupt or dropped data is detected rather than silently used. Full spec: `comms/PROTOCOL.md`.
- **No dependency in the other direction.** If the cable is unplugged or the Pi reboots, the ESP32 keeps sensing and braking. The Pi treats 2 seconds without a heartbeat as 'sensor link down'.

Who owns what

Member	Role	Owens in the repository
M1	Chassis & ergonomics	cad/, sensor mounts, camera mounting angle
M2	Actuation & power	Solenoid/MOSFET wiring, battery, fuse, harness (docs/WIRING.md)
C1	Embedded firmware	firmware/
C2	Computer vision	vision/pi/, training/
C3	Integration & fusion	comms/, fusion/, emergency/ (fall detection, GPS/GSM)
C4	Speech, test & docs	speech/, test/, README, final demo

3. What is inside the repository

Folder	What it does	Runs on
firmware/esp32/	PlatformIO firmware: sensor drivers, hard safety override, brake, haptics, UART. Pure-logic parts have host unit tests	ESP32-S3
firmware/esp32_arduino_ide/	The same firmware flattened into an Arduino IDE sketch (the IDE needs one flat folder), plus a blink test to verify your toolchain first	ESP32-S3
vision/pi/	Camera input, people/object detection, pothole detection, OCR, and <code>main.py</code> that ties everything together	Raspberry Pi
fusion/	State machine (IDLE, WALKING, HAZARD_WARNING, BRAKE_ENGAGED, FALL_ALERT) with an explicit priority order	Raspberry Pi
speech/	Fixed alert phrases, offline text-to-speech with latency logging, a watchdog buzzer that fires even if speech hangs	Raspberry Pi
comms/	UART protocol spec and the Pi-side implementation, a 1000-message link test, and a simulated sensor feed for hardware-free development	Pi (+ ESP32 side in firmware)
emergency/	Fall to SMS chain: GPS (NMEA) parser that only trusts a valid recent fix, SIM800L AT-command driver, cancel-by-grip countdown, no-repeat-spam policy, and a staged bench script	Raspberry Pi
training/	Dataset cleaning, YOLOv8 training, ONNX export and deployed-path evaluation for the pothole model	Laptop / PC
test/	Bench logger (live dashboard + labelled CSV), false-positive analyser, field test plan	Laptop / Pi
docs/	Architecture, wiring map, datasheet gotchas, BOM, Pi 3B setup guide, team roles	-

Quality measures built in

- **Tests for everything that can be tested without hardware:** 163 Python tests, plus C++ tests for the firmware's threshold and UART logic (19 of them, run on every push by GitHub's CI).
- **Simulation mode:** `main.py --sim` fakes the ESP32's sensor stream so the whole Pi side can be developed and demoed before any wiring exists.
- **Noise filtering at both layers:** a ground drop-off must be seen in two consecutive readings before the firmware brakes for it; vision needs a detection in 2 of the last 3 inferences before warning.
- **Solenoid protection:** the brake is force-released after 5 s regardless of what asked for it, because solenoids overheat if held on.
- **Self-checking:** the firmware logs how long its decide-and-engage step takes and warns if that exceeds 50 ms. That times the software step only, so prove the full sensor-to-brake time with a logic analyser or oscilloscope.

4. The AI models

SENSEWALK uses two neural networks and one classical OCR engine. Only one of them was trained by us, because the others are already good at their job; the gap was the hazard no public model knows: potholes.

4.1 People and objects: pretrained, verified (not trained by us)

Google's quantized MobileNet-SSD (COCO, 90 classes) running through the TFLite runtime. It is the default because it is small and was the one backend we could verify end to end with real weights. It tells the system *what* is ahead; it never estimates distance.

Test photo	Result	Note
Street scene with a bus	3 people found correctly, plus 1 false 'airplane'	Inference about 24 ms on a laptop CPU
Photo with 2 real people	5 person detections	Duplicate / extra boxes at the 0.5 threshold. The model is old and noisy

Fix for the noise: a person only counts as 'nearby' if their box is at least 45% of the frame height (a crude size-based proxy, since the camera has no depth), and only after 2 of the last 3 frames agree. Before this fix, a person 30 m away would have triggered speech and vibration. Treat 45% as a starting point to tune on real footage.

4.2 Potholes: the model we trained

COCO has no 'pothole' class, and potholes are the walker's headline hazard, so we fine-tuned a YOLOv8n network on public pothole photos. All numbers below are from images the model never saw during training or checkpoint selection.

Item	Value
Data after cleaning	913 unique images: 730 train / 91 validation / 92 test (1,959 labelled potholes)
Model	YOLOv8n fine-tuned from COCO weights, 320 x 320 input, ONNX 11.6 MB
Held-out test, deployed path	AP50 0.51 ; at confidence 0.35: precision 0.64 , recall 0.49 (91 hits, 51 false alarms, 96 misses)
Same weights, ultralytics' own scorer	mAP50 0.546 (the small gap is a different scoring convention)
Portability check	Identical results under OpenCV 4.8.1, 4.10.0 and 5.0.0
Speed on a laptop CPU	about 32-36 ms per frame; Pi 3B estimated roughly 10x slower (measure it)

In plain words: it spots roughly half of the potholes it is shown, and about one detection in three is wrong. That makes it a useful 'something odd on the ground ahead' nudge, not a safety device. This is exactly why it can only produce a spoken/vibration warning and can never trigger the brake.

Why you can trust the evaluation (and its limits)

- **No train/test leakage.** Public datasets contain several augmented copies of one photo. If one copy is in training and another in testing, the score is inflated. Images were clustered by visual similarity and each cluster kept in one split only.
- **The deployed code was scored, not just the training tool.** The ONNX file runs through our own pre-processing and box decoding on the Pi. Scoring that exact path means a bug there would have shown up as a collapse in accuracy.
- **Bad data was cleaned.** One public source had non-label text (someone's personal letter) inside a label file. It was dropped, and kept out of the repository. 101 exact duplicate images across sources were also removed.

Known weaknesses

- **Small, web-sourced data** (about 900 photos, mostly from car or standing height in daylight). A walker's camera is lower and sees pavements, floors and ramps.
- **No clean-road examples.** Every training image contains a pothole, so shadows, cracks and manhole covers were never taught as 'not a pothole'. Expect false alarms on them.
- **One class only.** Stairs, curbs and open drains are not detected yet.
- **92 test images** means the true accuracy could differ by about 10 points either way.
- **Training was interrupted** at epoch 24 of 60 when a session ended; the best checkpoint (around epoch 17) was used. The curve had flattened, but a full run might gain a little.

4.3 Reading signs: Tesseract OCR (not trained by us)

OCR only runs when the camera sees a sign-like rectangle, so it does not slow every frame. The region-finding and image clean-up code is tested on synthetic images; **its real-world accuracy on actual signs has not been measured**. The learning roadmap's test (10 signs, varied light and angle) still needs to be done and reported honestly.

4.4 How to make the pothole model better (the highest-value next step)

- **Film your own footage** with the real Camera Module 3 at the walker's real height: pavements, corridors, ramps, potholes, stairs, curbs.
- **Label it** (Roboflow, CVAT or Label Studio, YOLO export) and add it as another source to `training/prepare_dataset.py`.
- **Add hundreds of obstacle-free frames** with empty labels so it learns what is *not* a hazard.
- Add classes (stairs, curb, open_drain), retrain with `training/train_hazard_model.py`, and re-score with `training/evaluate_onnx.py`. Quote the new real numbers.

5. How the walker decides what to do

The fusion state machine on the Pi merges every input into one state, using a fixed priority so that physical safety always outranks information.

Priority	Input	Result	Can it brake?
1	Ground drop-off (downward ToF distance jumps)	BRAKE_ENGAGED, 'Pothole ahead, stopping'	Yes (real distance)
2	Obstacle close (ultrasonic or forward ToF)	BRAKE_ENGAGED, 'Obstacle close, stopping'	Yes (real distance)
3	Pothole seen by the camera	HAZARD_WARNING, 'Hazard on the ground ahead, slow down'	No, warning only
4	Person nearby (by apparent size)	HAZARD_WARNING, 'Person close ahead'	No, warning only
5	Sign read by OCR	WALKING, speaks the text	No, information
Always first	Fall detected (IMU)	FALL_ALERT, SMS with GPS location	-

- **Brake release needs a hand on the grip.** The brake does not let go just because the hazard cleared; the piezo grip sensors must confirm the user is holding on.
- **Even a brake decision is independent of the Pi.** The ESP32 applies its own copy of these brake rules directly, so the Pi's version is for user-facing alerts, not the safety path.
- **Spoken alerts are short fixed phrases**, not generated sentences, so latency is low and predictable, and a piezo buzzer fires on its own thread if speech stalls.

Fall alerts: how an SMS actually gets sent

- **A detected fall starts a 10-second countdown** ('Fall detected. Hold the handle to cancel the alert'). Gripping the handle cancels it; otherwise the SMS is sent. This exists because the fall rule is known to fire on things like sitting down hard, and texting family on false alarms teaches them to ignore the real one.
- **The message contains a map link only if the GPS has a valid fix less than 2 minutes old.** Otherwise it says 'location unavailable'. A stale position could send helpers to the wrong place.
- **A failed send is retried** after 30 s; only a successful send suppresses repeats (5 minutes), so one fall cannot produce a stream of texts and a brief network drop cannot swallow the alert.
- **Phone numbers never live in the code.** They come from the SENSEWALK_EMERGENCY_NUMBERS environment variable or --emergency-number, so they cannot be pushed to a public repository by accident.

Where the numbers come from

Every threshold (150 mm drop-off, 40 cm obstacle, grip level, fall impact) is a **starting guess** clearly tagged TODO(calibrate) in `firmware/esp32/include/config.h` and `fusion/state_machine.py`. The learning roadmap is explicit that these must be replaced by measurements (50+ test walks, 15+ controlled falls). Until that happens, the walker's hazard behaviour is untuned, and no accuracy claim about it can be made.

6. How to implement this in the real project

This is the order that keeps the team unblocked. Each step has a concrete 'done when' so progress is visible, and each ties to the 8-week plan in the research report.

When	Who	Do this	Done when
Week 1 (before ordering)	Everyone	Clone the repo, run <code>python -m pytest</code> . Each person writes a one-page datasheet 'gotcha sheet' for their parts (starter: <code>docs/DATASHEET_GOTCHAS.md</code>)	163 tests pass on every laptop; gotcha sheets shared
Week 1	C1	Install Arduino IDE; flash <code>SENSEWALK_blink_test</code> , then check the Serial Monitor	LED blinks and serial text appears
Week 1	C2 / C3	Flash Raspberry Pi OS Lite 64-bit, run <code>sudo apt full-upgrade</code> , then <code>libcamera-hello --list-cameras</code>	Camera Module 3 is listed and a still image saves
Phase 1 (days 1-10)	C1 + M2	Wire ToF, ultrasonic, grips per <code>docs/WIRING.md</code> . Flash the full sketch. Walk test scenarios while <code>test/bench_logger.py</code> records labelled CSVs	50+ walks over a drop and 50+ over flat ground logged
Phase 1	C1	Run <code>test/analyze_detection_log.py</code> ; replace every <code>TODO(calibrate)</code> value in <code>config.h</code> with measured ones	False-positive and false-negative rates computed and quoted
Phase 2 (11-20)	M1, M2, C1	Build chassis; wire MOSFET + flyback diode + fuse; test the solenoid brake 20 times in a row. Pair M2 with C1 here	Brake engages in <50 ms (measured with a logic analyser or scope), with the Pi unplugged
Phase 3 (21-32)	C2, M1	On the Pi: install deps, download the model, run <code>main.py --sim --preview</code> , then measure FPS with different flags. Decide camera height/angle together with M1	Real FPS and temperature logged for 10 minutes
Phase 3	C2	Film footage on the real walker; label it; retrain the pothole model; re-score it	New held-out numbers in the report
Phase 3	C4	Speaker + PAM8403 wiring; log trigger-to-sound latency for 5+ phrases; kill TTS mid-run to confirm the buzzer fires	Latency table done; buzzer proven
Phase 4 (33-44)	C3 (+ all)	Run the 1000-message heartbeat test, then connect real sensors. Calibrate fall detection with 15+ padded drops. Run <code>python -m emergency.send_test_sms outdoors</code> for a real GPS fix and one real SMS (SIM800L on its own ~2 A supply, never a logic pin)	0 lost messages; one SMS with coordinates received
Phase 5 (45-56)	C4 (+ all)	Run the written field-test plan: braking distance, false alerts per 100 m, battery life; each with 10+ trials. Rehearse the demo; record a backup video	Results CSVs and mean/spread numbers in the report

Commands you will actually use

On your laptop (everyone):

```
git clone https://github.com/sayansaunth-lang/sensewalk
cd sensewalk
python -m venv venv && venv\Scripts\activate      # Windows
pip install -r requirements-dev.txt
python -m pytest                                # expect: 163 passed
```

ESP32-S3 (C1), Arduino IDE: follow `firmware/esp32_arduino_id/README.md`. Add the Espressif board URL, install the libraries *VL53L1X* (Pololu) and *Adafruit MPU6050*, choose 'ESP32S3 Dev Module', then upload the blink test first and SENSEWALK_ESP32 second. Edit pins in `config.h` to match your wiring, not the other way round.

Raspberry Pi 3B (C2): full walkthrough in `docs/PI3B_LOW_RAM_SETUP.md`.

```
sudo apt install python3-picamera2 --no-install-recommends
python3 -m venv --system-site-packages venv      # flag is REQUIRED for picamera2
source venv/bin/activate
pip install -r vision/pi/requirements.txt
pip install tfllite-runtime
bash vision/pi/models/download_tfllite_model.sh  # checksum-verified

# no ESP32 yet? fake its sensors:
python3 vision/pi/src/main.py --camera picamera2 --sim --preview \
    --process-every-n-frames 3 --width 320 --height 240

# pothole model + real sensors + fall alerts (numbers come from the environment, never from code):
export SENSEWALK_EMERGENCY_NUMBERS='+91XXXXXXXXXX'
python3 vision/pi/src/main.py --camera picamera2 --hazard-model \
    --serial-port /dev/serial0 --process-every-n-frames 4 \
    --gsm-port /dev/ttyUSB1 --gps-port /dev/ttyUSB0
```

Retrain the pothole model (C2, on a PC): `training/README.md` has the four commands, including how to finish an interrupted run.

Pi 3B tuning in one sentence: keep the camera at 320 x 240, run detection on every 3rd or 4th frame, and measure FPS with `--preview` before believing any number. A working walker at 2-3 detections per second with an honest write-up is a better result than an unverified claim of 12-15 FPS.

7. What was tested, and what was not

Verified

What	How	Result
Pi-side logic (UART codec, fusion, speech, vision helpers, dataset tool, evaluators)	163 automated unit tests on a laptop	All pass
People/object detector with real weights	Run on real photos (bus scene, 2-person photo)	Works; some false detections
Pothole model, deployed path	92 held-out images, OpenCV 4.8.1 / 4.10.0 / 5.0.0	AP50 0.51, identical across versions
Whole chain on real photos	Photo to detectors to confirm filter to fusion to spoken phrase	Pothole photos say 'Hazard on the ground ahead'; people photos say 'Person close ahead'; blank frame stays silent
ESP32 firmware compiles	GitHub CI: PlatformIO build, and both Arduino IDE sketches with the latest ESP32 core	Passes (after 2 compile errors were found and fixed)
Firmware logic and UART codec	19 C++ unit tests on the CI machine	All pass; the C++ checksum matches the Python one byte for byte
Model downloads	Fetches from the official hosts; TFLite zip checksum pinned	Intact; one earlier link was dead and was removed

Problems found and fixed while verifying (why testing on real inputs mattered)

Problem	How it was found	Fix
No GPS/GSM code existed, although fall-to-SMS is a headline feature	Searching the code before writing this report	Built emergency/ (section 3)
Pipeline crashed with OCR on when Tesseract was missing	Ran the real <code>main.py</code> over a video	Error normalised; OCR turns itself off with one warning
First sign read would have crashed (speech template missing its text)	Reading the call path against the phrase table	Pass the OCR text; regression test added
A fall would latch the Pi in FALL_ALERT forever (the ESP32 only ever sends fall=1)	Tracing the message flow end to end	Time-limited latch before the countdown
Any person at any distance triggered warnings	Reviewing detector output on real photos	Size-based 'nearby' test plus 2-of-3 confirmation
SMS driver sent the message body after the modem answered ERROR	A unit test against a scripted fake modem	Check for the '>' prompt before sending
Firmware did not compile (struct initialisation; a missing include)	First GitHub CI build of the firmware	Fixed in both firmware copies; CI now builds both
UART protocol: encoder and decoder disagreed on field length; checksum could read past its buffer	A failing C++ unit test on CI	Both ends enforce the same limits; buffer read clamped; tests on both sides
Caffe model download link was dead (404)	Actually downloading it	TFLite made the default; Caffe marked unsupported
picamera2 not importable in a normal Python venv on the Pi	Re-reading the setup steps	Create the venv with <code>--system-site-packages</code>
PyTorch wheel arrived corrupted twice on this network	pip's hash check failed	Resumable download with SHA-256 verification; checksum pinned in the model script

NOT verified (do not assume these work)

Item	Why it matters
ESP32 firmware running on a real board	It compiles, but flashing, sensor start-up and the FreeRTOS timing have never been seen on hardware
Anything on the Pi 3B	Camera Module 3 capture, real FPS, thermal behaviour, text-to-speech audio, buzzer GPIO
All physical sensors and actuators	ToF address setup, ultrasonic, IMU, grips, solenoid brake latency, haptics
UART link between the boards	The 1000-message test exists but has not been run on hardware
OCR on real signs; real GPS fix; real SIM800L SMS	Only the surrounding logic is tested (the SMS chain against a scripted fake modem)
Timing: brake engages within 50 ms	Only a software-step timer exists; needs a logic analyser or oscilloscope on the real brake

8. Risks and open issues

Risk / issue	What to do
Placeholder thresholds can cause missed hazards or constant false braking	Calibrate in Phase 1 with the bench logger before any walking test with a person
Pi 3B may be too slow for both models at once	Use frame skipping; drop the pothole model for a demo if needed. The ESP32 safety layer is unaffected
Pothole model false alarms on cracks and shadows	Add clean-road negatives and your own footage (section 4.4)
Downloads can arrive corrupted on this network	Large files failed silently here once. Verify checksums (the TFLite script does)
SMS may never send: SIM800L is 2G-only and some carriers have retired 2G; it also resets if powered from a logic pin	Test your SIM in a phone forced to 2G first; power the module from its own ~2 A supply (docs/WIRING.md)
The Pi 3B has one usable hardware UART (used by the ESP32)	Connect GPS and GSM through two USB-to-TTL adapters (about Rs 300, added to the BOM)
Licence: YOLOv8 is AGPL-3.0	Fine for a university project; decide before any distribution or commercial use
Solenoid, battery and mains-adjacent wiring are physical-safety items	Fuse every 12 V line, check wiring with a teammate before first power-on, never power the SIM800L from a logic pin

9. Recommended next steps, in order

1. Pull the latest repo, run the tests, and flash the **blink test** on the ESP32-S3. This proves your tools work before anything else depends on them.
2. Flash the full firmware with no sensors attached. It already compiles in CI, so what you are checking is that it halts with a clear message when the ToF sensors are missing, which proves the safety check fires on real hardware.
3. Boot the Pi 3B headless, confirm `libcamera-hello` sees the Camera Module 3, and run `main.py --sim --preview`. Record the real FPS.
4. Wire the ToF and ultrasonic sensors and run the bench logger. Replace the placeholder thresholds with measured ones.
5. Order and wire the brake driver, then prove the under-50 ms engage with the Pi disconnected.
6. Before relying on fall alerts, test your SIM on 2G, buy two USB-to-TTL adapters, and run `python -m emergency.send_test_sms` outdoors.
7. Start filming real pothole, step and curb footage with the actual camera now, since labelled data is the slowest part of improving the model.
8. Keep an eye on the green tick on GitHub: every push re-runs all tests and both firmware builds, so a broken change shows up immediately.

Appendix - Licences and sources

Component	Source	Licence
People/object model	Google TensorFlow Lite example: <code>coco_ssd_mobilenet_v1_1.0_quant</code>	Apache-2.0 (TensorFlow)
Pothole base network	Ultralytics YOLOv8n (COCO-pretrained)	AGPL-3.0
Pothole data (1)	Hugging Face: <code>rupesh002/pothole-detection-dataset</code> (a Roboflow export)	MIT per listing; original terms unverified
Pothole data (2)	Hugging Face: <code>Ryukijano/Pothole-detection-Yolov8</code> , Roboflow workspace <code>project-ssayl</code> , <code>'potholes-detection-d4rma'</code> v1	CC BY 4.0 (attribution required)
OCR engine	Tesseract	Apache-2.0

The raw datasets are not stored in the repository. The trained model is `vision/pi/models/hazard_yolov8n_320.onnx`, and the full training record is in `training/README.md`.